# EEEG513 — Machine Learning

## Part B: Open-Book Coding Exam

Marks Prediction

**Duration:** 2 hours
**Total Marks:** 79
**Resources:** Open book, open laptop, open AI
**Data:** `./25_26_1.csv/ and ./25_26_2.csv`

---

### Rules

1. You may use any Python library for data processing and feature engineering.
2. The **final prediction model** must be one of the three specified types. Using a different model as the final predictor results in disqualification for that component.
3. No communication with other students during the exam.
4. All code and predictions must be your own work.
5. Any plagiarism will result in straight 0. Your reports and code files will be scrutinized to detect plagiarism.
6. Predictions are automatically clipped to $[10^{-6},\ 1 - 10^{-6}]$ before scoring.

### Overview

You have just completed Part A, a 24-question closed-book exam (26 marks total) with 4 sections. Your task is to **predict the probability that each student answers each question correctly**, using their past evaluative performance as features.

You will build **three separate models** to make the same prediction. Each model is scored independently.

### Dataset

All data is organised into two subfolders:

- `./25_26_1/` — **Last semester** (training data): component-wise marks for all students, plus all question papers from that semester.
- `./25_26_2/` — **This semester** (training data): component-wise marks for all students *up to but not including* Part A compre, plus all question papers from this semester.

#### Feature Data

The tabular dataset includes (for both semesters): quiz scores, midterm scores, coding exam scores, lab scores, project scores, attendance, and any other component-wise evaluations conducted during the semester.

#### Question Papers

All question papers for both semesters are provided as PDFs in their respective folders. You may inspect these to understand topic coverage and difficulty across assessments. **Last semester's Part A answer key is not provided** — you must infer topic-level performance from total scores and the question papers yourself.

### Course Topics

The course covers 8 topic areas:

| ID | Topic |
|---|---|
| T1 | Logistic / Linear Regression |
| T2 | Probability / Naïve Bayes / MLE |
| T3 | SVM |
| T4 | Decision Trees / Ensemble Methods |
| T5 | Perceptron |
| T6 | Neural Networks / Backprop / CNN |
| T7 | Unsupervised Learning / Clustering |
| T8 | Gradient Descent / Optimization |

## Prediction Target

Today's Part A exam had **24 questions** (26 marks; Q9 and Q21 are worth 2 marks each) across **4 sections**. Your prediction target is a **24-dimensional probability vector** per student:

$$\hat{\mathbf{p}}_i = \Big[\ P(Q_1 = 1 \mid \text{student } i),\ \ P(Q_2 = 1 \mid \text{student } i),\ \ \ldots,\ \ P(Q_{25} = 1 \mid \text{student } i)\ \Big]$$

where $P(Q_j = 1 \mid \text{student } i) \in [0, 1]$ is your predicted probability that student $i$ answers question $j$ correctly.

### Feature Data

The tabular dataset contains the following groups of features for each student:

**Evaluative Scores:** `Test 1`, `Test 2`, and `Coding Test` are aggregate scores on the three assessments conducted before Part A. `Lab Attendance` is self-explanatory. `Project Midsem`, `Project compre`, and `Project Total` are the project evaluation scores, with a further breakdown into three rubric dimensions — overall idea (6 marks), depth of research (15 marks), and code awareness (9 marks) — for both the midsem and compre evaluations. `Team Number` identifies the project group.

**Topic-Wise Test Scores:** Columns prefixed with `T1-` are per-topic breakdowns of Test 1, columns prefixed with `T2-` are per-topic breakdowns of Test 2, and columns prefixed with `CT-` are per-topic breakdowns of the Coding Test. For example, `T1-LR` is the student's score on the Logistic Regression portion of Test 1, `T2-CNN` is the score on the CNN portion of Test 2, and `CT-Backprop` is the score on the Backpropagation portion of the Coding Test. These are your most granular features and can be mapped directly to Part A questions using the topic tags provided in the question map.

## Part A Question Map — Topic & Difficulty

Each question below is tagged with its **course topic**, **section**, and **estimated difficulty**. Use this information to connect student features to question-level predictions.

| Q | Topic | Section | Difficulty | Marks |
|---|---|---|---|---|
| 1 | MLE vs MAP convergence | A | Hard | 1 |
| 2 | SVM / Kernel Trick | A | Easy | 1 |
| 3 | Naïve Bayes zero probability | A | Easy | 1 |
| 4 | Logistic loss behaviour | A | Medium | 1 |
| 5 | Log. Reg. weight divergence | A | Hard | 1 |
| 6 | Naïve Bayes assumption | A | Easy | 1 |
| 7 | Entropy of a pure node | B | Easy | 1 |
| 8 | Information gain definition | B | Easy | 1 |
| 9 | Information gain calculation | B | Hard | 2 |
| 10 | Overfitting / pruning | B | Easy | 1 |
| 11 | Majority voting condition | B | Medium | 1 |
| 12 | Ensemble variance | B | Medium | 1 |
| 13 | ReLU derivative | C | Easy | 1 |
| 14 | NAND gate (perceptron) | C | Easy | 1 |
| 15 | Sigmoid derivative | C | Medium | 1 |
| 16 | CNN false statement | C | Medium | 1 |
| 17 | ResNet / skip connections | C | Hard | 1 |
| 18 | Backprop chain rule | C | Hard | 1 |
| 19 | AdaGrad update rule | D | Medium | 1 |
| 20 | SGD vs batch GD | D | Medium | 1 |
| 21 | Convergence from eigenvalues | D | Medium | 2 |
| 22 | Learning rate too large | D | Easy | 1 |
| 23 | Condition number effect | D | Hard | 1 |
| 24 | Underfitting diagnosis | D | Hard | 1 |

**Difficulty summary:** 9 Easy, 8 Medium, 8 Hard. **Total:** 24 questions, 26 marks.

### Topic Grouping

For mapping to the 8 course topics:

**T1 — Logistic/Linear Regression:** Q4, Q5, Q24
**T2 — Probability/Naïve Bayes/MLE:** Q1, Q3, Q6
**T3 — SVM:** Q2
**T4 — Decision Trees/Ensembles:** Q7, Q8, Q9, Q10, Q11, Q12
**T5 — Perceptron:** Q14
**T6 — Neural Networks/Backprop/CNN:** Q13, Q15, Q16, Q17, Q18
**T7 — Unsupervised Learning:** None
**T8 — Gradient Descent/Optimization:** Q19, Q20, Q21, Q22, Q23

## Models to Build

You must build exactly three models. Each model independently predicts the same target (25 probabilities per student). You may use different features, preprocessing, or training strategies for each.

| Part | Model | Constraints |
|---|---|---|
| A | Linear Regression | Standard linear/logistic regression. Regularisation (L1/L2) is permitted. You may train one model per question, per section, or a single multi-output model. |
| B | Neural Network | Feed-forward network with **at most 3 hidden layers** (input and output layers do not count). Any activation function. |
| C | Random Forest | Scikit-learn `RandomForestRegressor` or `RandomForestClassifier`. You may tune hyperparameters freely. |

You may **not** use any other model type as the final predictor (no XGBoost, no SVM regression, etc.). However, you are free to use any method for feature engineering, feature selection, or preprocessing.

## Evaluation

Your predictions will be scored using **log-loss** (binary cross-entropy) averaged across all questions and students:

$$\mathcal{L} = -\frac{1}{N \times 24} \sum_{i=1}^{N} \sum_{j=1}^{24} \Big[ \, y_{ij} \ln(\hat{p}_{ij}) + (1 - y_{ij}) \ln(1 - \hat{p}_{ij}) \, \Big]$$

where $y_{ij} \in \{0, 1\}$ is whether student $i$ answered question $j$ correctly, and $\hat{p}_{ij} \in [\epsilon,\, 1-\epsilon]$ is your predicted probability (clipped to $\epsilon = 10^{-6}$ to avoid $\ln(0)$).

Each model (LR, NN, RF) is scored independently:

| Component | Marks |
|---|---|
| Linear / Logistic Regression (log-loss) | 22 |
| Neural Network (log-loss) | 22 |
| Random Forest (log-loss) | 22 |
| Report | 13 |
| **Total** | **79** |

Marks within each component are assigned based on your rank in the class: the lowest log-loss gets full marks, scaled down to a minimum for the highest log-loss. The exact scaling will be announced after grading.

## Deliverables

### Prediction Files (3 CSV files)

Each file must contain exactly $N$ rows (one per this-semester student) and 26 columns:

| Column | Type | Description |
|---|---|---|
| `student_id` | string | Anonymised student identifier (as in the dataset) |
| `Q1` | float $\in [0, 1]$ | $P(\text{Q1 correct} \mid \text{student})$ |
| `Q2` | float $\in [0, 1]$ | $P(\text{Q2 correct} \mid \text{student})$ |
| $\vdots$ | $\vdots$ | $\vdots$ |
| `Q25` | float $\in [0, 1]$ | $P(\text{Q25 correct} \mid \text{student})$ |

Name your files as follows (replace `<ID>` with your roll number):

```
<ID>-LR.csv (Linear / Logistic Regression predictions)
<ID>-NN.csv (Neural Network predictions)
<ID>-RF.csv (Random Forest predictions)
```

### Code (3 Python scripts)

Submit the scripts used to train each model and generate predictions:

```
<ID>-LR.py
<ID>-NN.py
<ID>-RF.py
```

Each script must run end-to-end and produce the corresponding CSV file. **Code that does not compile will receive zero marks for that component.**

### A Report Detailing you Approach

Document your entire approach in a PDF for all the 3 models. It can be easily generated using AI and your code file.

### Submission

Upload all 7 files (3 CSVs + 3 scripts + 1 Report) to Quanta before the exam ends.

## Suggested Approach

These are not mandatory, but thinking about them will improve your predictions.

**Hints**

1. Since labels for training the model are not directly provided, your first should be to create a 'test' set or a proxy that you can use to train your model. It can be last year's questions and answers or this semester's previous test scores.

2. **Topic-tag last semester's assessments.** Read last semester's question papers and categorise each question by the 8 course topics. This lets you construct per-topic scores from the raw marks — far more predictive than aggregate totals.

3. **Model questions jointly, not independently.** A student who gets Q7 right (easy entropy) is more likely to get Q9 right (hard IG calculation) — both test decision tree knowledge. Shared latent factors ("decision tree ability", "optimization ability") can help. Think about whether you can model a student's latent topic-level ability and map it to per-question probabilities.

4. **Hyperparameter tuning**: After you have finalized your features, tune hyperparameters of models heavily to extract maximum performance.